```
#Gauss Elimnination
import numpy as np
import sys

def gauss_elimination(A):

    n = A.shape[0]  # Number of unknowns (and rows)

    # Elimination Phase
    for i in range(n):
        # Find pivot element (largest absolute value in the current column below the diagonal)
       """ max_row = i
        for k in range(i + 1, n):
            if abs(A[k, i]) > abs(A[max_row, i]):
                max_row = k

        # Swap rows to bring the pivot to the current diagonal position
        A[[i, max_row]] = A[[max_row, i]]
"""
        # Check for division by zero (singular matrix)
        if A[i, i] == 0:
            print("Divide by zero detected or singular matrix. No unique solution exists.")
            return None

        # Eliminate elements below the current pivot
        for k in range(i + 1, n):
            factor = A[k, i] / A[i, i]
            A[k, i:] = A[k, i:] - factor * A[i, i:]

    # Backward Substitution Phase
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        if A[i, i] == 0:  # Check for zero on diagonal during back substitution
            print("Singular matrix detected during back substitution. No unique solution exists.")
            return None
        x[i] = A[i, n] / A[i, i]
        for k in range(i - 1, -1, -1):
            A[k, n] = A[k, n] - A[k, i] * x[i]
    return x

augmented_matrix = np.array([
        [1,2,3, 10],
        [1,3,4,40],
        [1,1,1,40]])

print("Original Augmented Matrix:")
print(augmented_matrix)

solution = gauss_elimination(augmented_matrix.copy()) # Use .copy() to avoid modifying original

if solution is not None:
    print("\nSolution:")
    print(solution)
```

```
Original Augmented Matrix:
[[ 1  2  3 10]
 [ 1  3  4 40]
 [ 1  1  1 40]]

Solution:
[ 10.  90. -60.]
```

```
#digonalization of matrix
import math
def multiply_matrices(A, B):
    result = []
    for i in range(len(A)):
        row = []
        for j in range(len(B[0])):
            sum = 0
            for k in range(len(B)):
                sum += A[i][k] * B[k][j]
            row.append(sum)
        result.append(row)
    return result
```

```
def inverse_2x2(matrix):
    det = matrix[0][0]*matrix[1][1] - matrix[0][1]*matrix[1][0]
    if det == 0:
        raise ValueError("Matrix is singular and cannot be inverted")
    return [[ matrix[1][1]/det, -matrix[0][1]/det],
            [-matrix[1][0]/det,  matrix[0][0]/det]]

def compute_eigen_2x2(matrix):
    a, b, c, d = matrix[0][0], matrix[0][1], matrix[1][0], matrix[1][1]

    # Characteristic Equation: λ² - (a+d)λ + (ad - bc) = 0
    trace = a + d
    determinant = a*d - b*c

    discriminant = trace**2 - 4*determinant

    if discriminant < 0:
        print("Complex eigenvalues, not supported here.")
        return None, None

    sqrt_disc = math.sqrt(discriminant)  #sqrt_disc = discriminant ** 0.5

    lambda1 = (trace + sqrt_disc) / 2
    lambda2 = (trace - sqrt_disc) / 2

    # Find eigenvectors manually (one for each eigenvalue)
    def find_vector(lam):
        # (A - λI)v = 0 -> Solve: (a - λ)v1 + bv2 = 0
        # Choose v2 = 1
        v1 = -(b) / (a - lam) if a - lam != 0 else 1
        return [v1,1]


    eigenvector1 = find_vector(lambda1)
    eigenvector2 = find_vector(lambda2)

    return [lambda1, lambda2], [eigenvector1, eigenvector2]


A = [[4, 1],
     [2, 3]]

eigenvalues, eigenvectors = compute_eigen_2x2(A)

print("Eigenvalues:", eigenvalues)
print("Eigenvectors:", eigenvectors)
P = [[eigenvectors[0][0], eigenvectors[1][0]],
     [eigenvectors[0][1], eigenvectors[1][1]]]

print("Matrix P:", P)
print(inverse_2x2(P))
#D=multiply_matrices(multiply_matrices(inverse_2x2(P),A),P)
D=multiply_matrices(inverse_2x2(P),multiply_matrices(A,P))
print(D)
```

```
Eigenvalues: [5.0, 2.0]
Eigenvectors: [[1.0, 1], [-0.5, 1]]
Matrix P: [[1.0, -0.5], [1, 1]]
[[0.6666666666666666, 0.3333333333333333], [-0.6666666666666666, 0.6666666666666666]]
[[5.0, 0.0], [0.0, 2.0]]
```

Demonstrate Discrete and Continuous Random Variables (Binomial and Normal Distributions)

```
import numpy as np
import matplotlib.pyplot as plt
import math
def factorial(n):
    return 1 if n == 0 else n * factorial(n-1)

def nCr(n, r):
    return factorial(n) // (factorial(r) * factorial(n-r))

def binomial_pmf(n, p):
    probs = []
    for k in range(n+1):
```

```
      prob = nCr(n, k) * (p**k) * ((1-p)**(n-k))
      probs.append(prob)
  return probs

n = int(input("Enter number of trials (n): "))
p = float(input("Enter probability of success (p): "))

x = np.arange(0, n+1)
pmf = binomial_pmf(n, p)

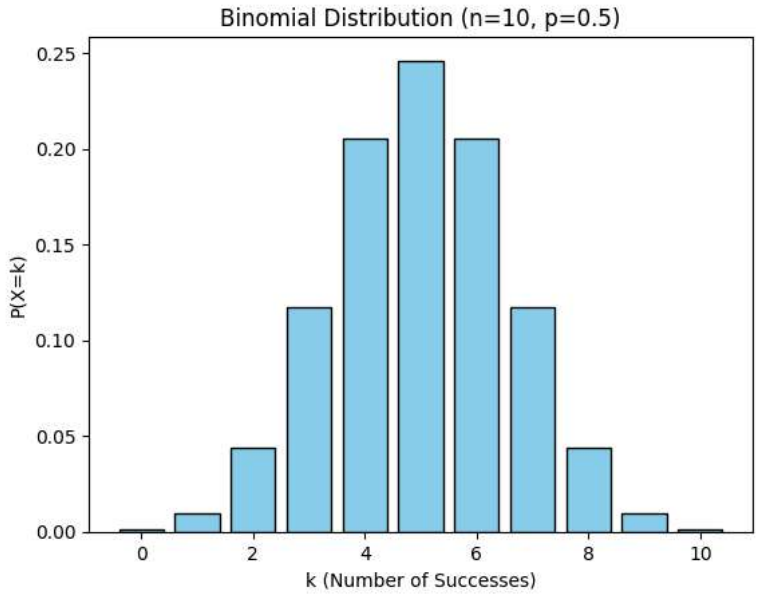
print(f"\nBinomial Distribution: n = {n}, p = {p}")
print("k\tP(X=k)")
for k, prob in zip(x, pmf):
    print(f"{k}\t{prob:.5f}")



# Binomial
plt.bar(x, pmf, color='skyblue', edgecolor='black')
plt.title(f"Binomial Distribution (n={n}, p={p})")
plt.xlabel("k (Number of Successes)")
plt.ylabel("P(X=k)")
plt.show()
```

```
Enter number of trials (n): 10
Enter probability of success (p): 0.5

Binomial Distribution: n = 10, p = 0.5
k   P(X=k)
0   0.00098
1   0.00977
2   0.04395
3   0.11719
4   0.20508
5   0.24609
6   0.20508
7   0.11719
8   0.04395
9   0.00977
10  0.00098
```



Demonstrate Normal Distribution

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

```
import math
import numpy as np
import matplotlib.pyplot as plt

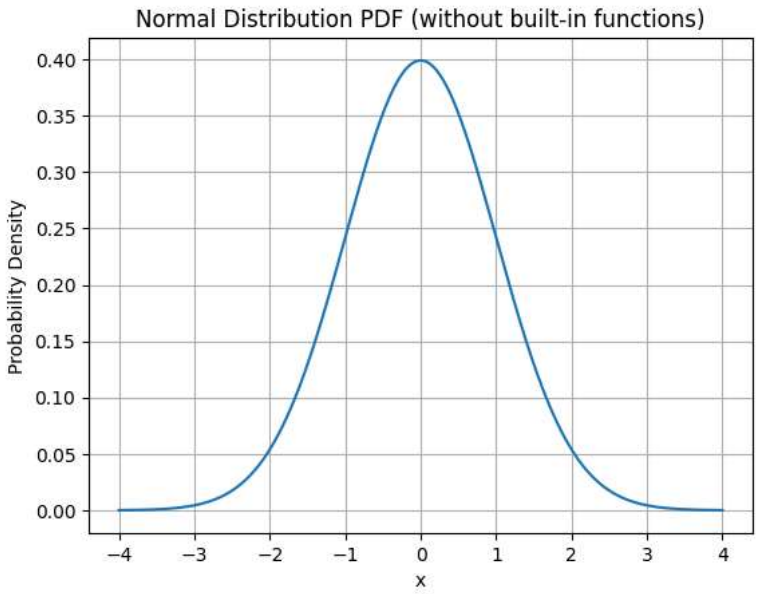
def normal_pdf(x, mu, sigma):
    coeff = 1 / (sigma * math.sqrt(2 * math.pi))
    exponent = -((x - mu) ** 2) / (2 * sigma ** 2)
    return coeff * math.exp(exponent)

# Parameters
mu = 0       # mean
sigma = 1    # standard deviation

# Generate x values
x_values = np.linspace(-4,4, 1000)

pdf_values = [normal_pdf(x, mu, sigma) for x in x_values]

# Plotting
plt.plot(x_values, pdf_values, label=f'N({mu},{sigma}²)')
plt.title('Normal Distribution PDF (without built-in functions)')
plt.xlabel('x')
plt.ylabel('Probability Density')
plt.grid(True)
plt.show()
```



Central Limit Theorem

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
```

```
# Step 1: Generate a non-normal population (e.g., exponential distribution)
#population = np.random.exponential(scale=2.0, size=10000)
population = np.random.uniform(0, 1, 100000)
# Step 2: Define sampling parameters
sample_size = 50      # number of elements in each sample
num_samples = 1000    # number of samples

# Step 3: Compute sample means
sample_means = []
for _ in range(num_samples):
    sample = np.random.choice(population, size=sample_size, replace=True)
    sample_means.append(np.mean(sample))

# Step 4: Plot distributions
plt.figure(figsize=(12,6))

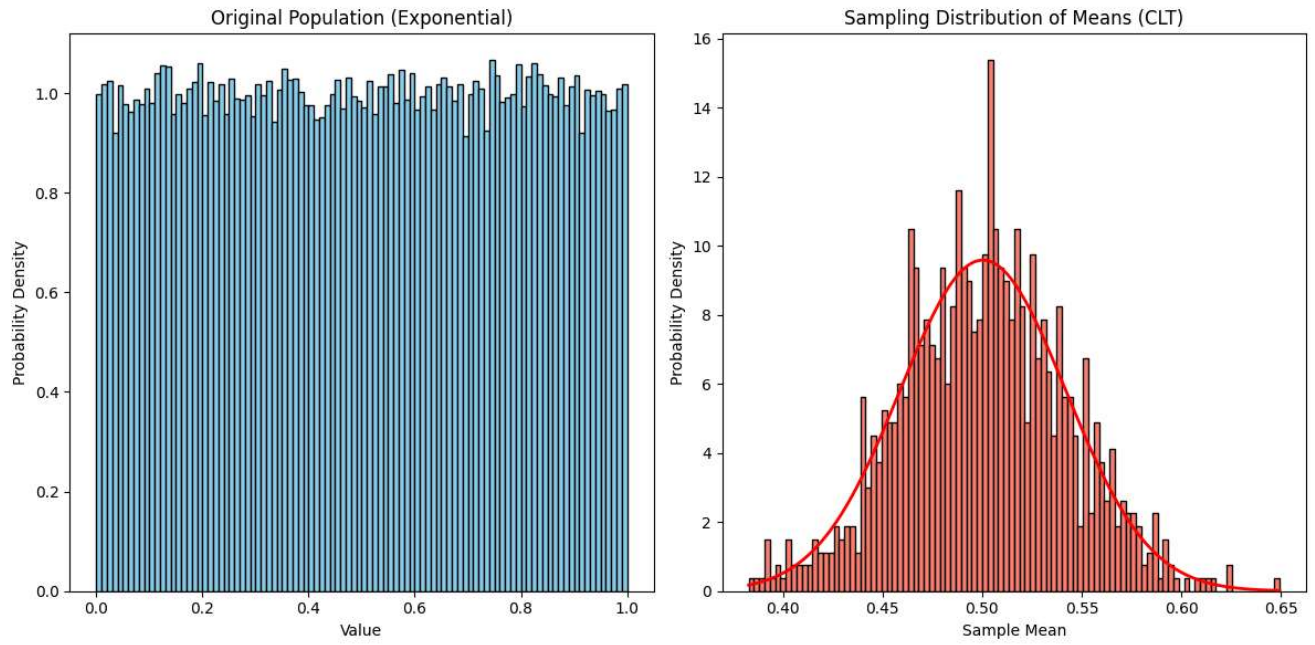
# Plot original population
plt.subplot(1,2,1)
plt.hist(population, bins=100, color='skyblue', edgecolor='black', density=True)
plt.title("Original Population (Exponential)")
plt.xlabel("Value")
plt.ylabel("Probability Density")

# Plot sampling distribution of means
plt.subplot(1,2,2)
plt.hist(sample_means, bins=100, color='salmon', edgecolor='black', density=True)
mean = np.mean(sample_means)
std = np.std(sample_means)

# Generate normal distribution curve
x = np.linspace(min(sample_means), max(sample_means), 1000)
pdf = norm.pdf(x, mean, std)

#Plot the normal curve
plt.plot(x, pdf, 'r', linewidth=2, label="Normal Distribution (CLT Prediction)")
plt.title("Sampling Distribution of Means (CLT)")
plt.xlabel("Sample Mean")
plt.ylabel("Probability Density")

plt.tight_layout()
plt.show()
```



T & F Distribution

$$T = \frac{Z}{\sqrt{V/df}}$$

where Z =N(0,1) and $V = \sum_{i=1}^{df} Z_i^2$

$$F = \frac{(U/d_1)}{(V/d_2)}$$

```
import random
import math
import matplotlib.pyplot as plt

# Standard Normal using CLT
def normal_random():
    return sum(random.random() for _ in range(12))-6  # approx N(0,1)

# t-distribution random variable
def t_random(df):
    z = normal_random()
    v = sum(normal_random()**2 for _ in range(df))  # sum of squared normals
    return z / math.sqrt(v / df)

# F-distribution random variable
def f_random(d1, d2):
    u = sum(normal_random()**2 for _ in range(d1)) / d1
    v = sum(normal_random()**2 for _ in range(d2)) / d2
    return u / v

# Generate many samples
def generate_samples(func, n, *args):
    return [func(*args) for _ in range(n)]

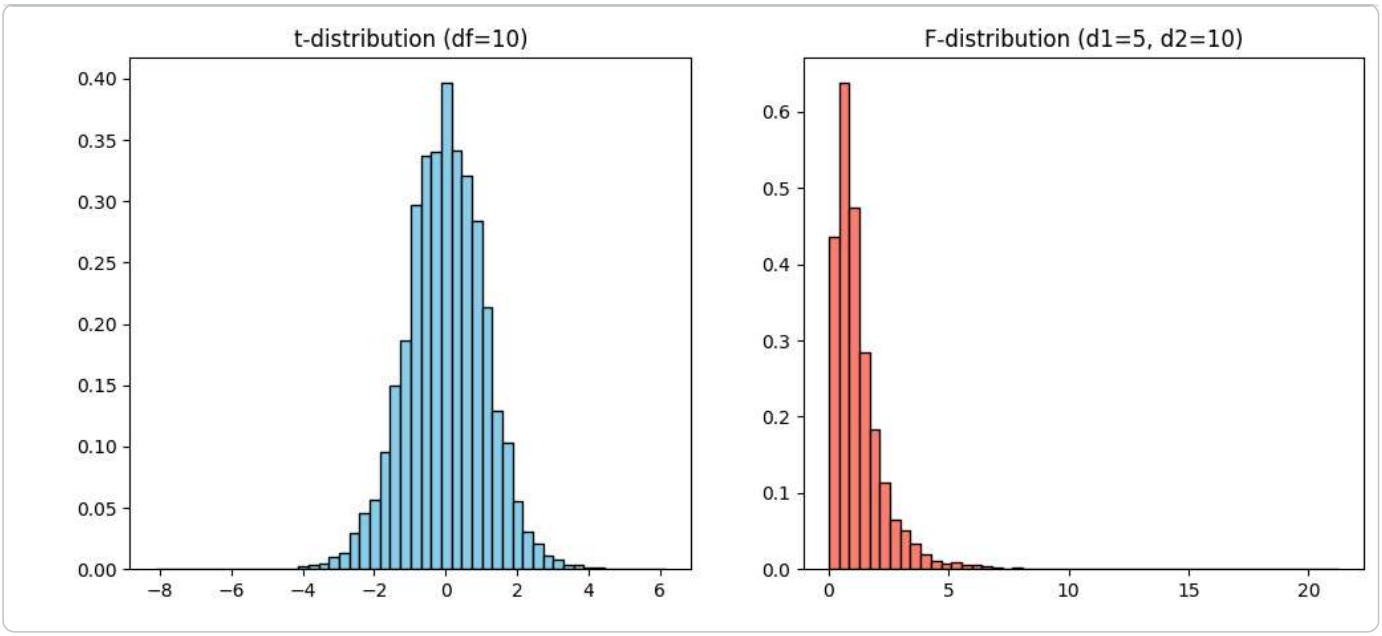
# Example usage
t_samples = generate_samples(t_random, 5000, 10)    # t with df=10
f_samples = generate_samples(f_random, 5000, 5, 10) # F with d1=5, d2=10

# Plot
plt.figure(figsize=(12,5))

# t-distribution
plt.subplot(1,2,1)
plt.hist(t_samples, bins=50, density=True, color="skyblue", edgecolor="black")
plt.title("t-distribution (df=10)")

# F-distribution
plt.subplot(1,2,2)
plt.hist(f_samples, bins=50, density=True, color="salmon", edgecolor="black")
plt.title("F-distribution (d1=5, d2=10)")

plt.show()
```



Joint Probability Mass Function

```
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# STEP 1: Define sample space
# ----------------------------
# Example: Roll of 2 dice
# Random variable X = outcome of Die 1
# Random variable Y = outcome of Die 2

die_outcomes = [1, 2, 3, 4, 5, 6]

# Total possible outcomes = 36
sample_space = [(x, y) for x in die_outcomes for y in die_outcomes]

# ----------------------------
# STEP 2: Compute Joint PMF
# ----------------------------
# Probability of each pair (x,y) is 1/36 since fair dice
joint_pmf = {}
for (x, y) in sample_space:
    joint_pmf[(x, y)] = 1/36

# ----------------------------
# STEP 3: Store probabilities in matrix form
# ----------------------------
pmf_matrix = np.zeros((6, 6))  # 6x6 matrix for dice

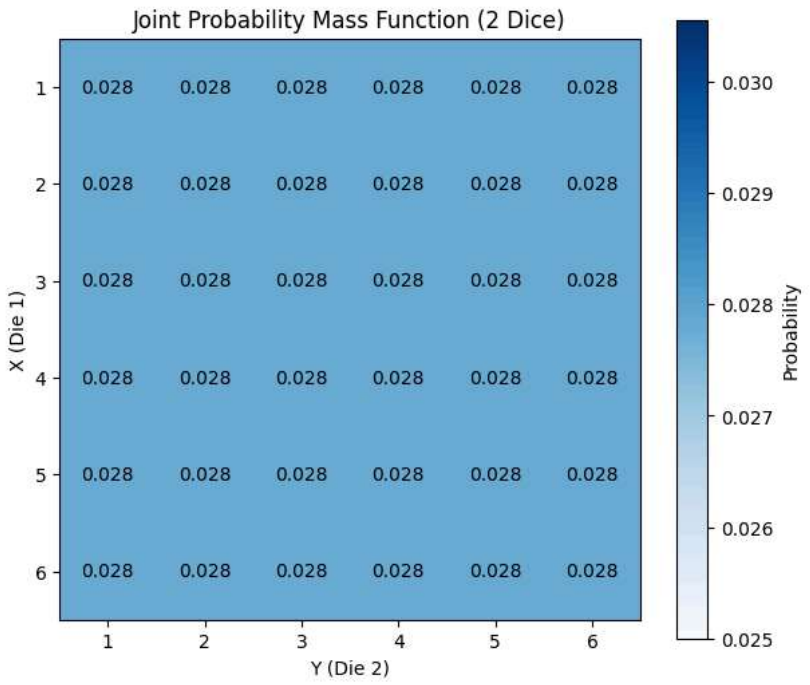
for x in die_outcomes:
    for y in die_outcomes:
        pmf_matrix[x-1, y-1] = joint_pmf[(x, y)]

# ----------------------------
# STEP 4: Visualization using heatmap
# ----------------------------
plt.figure(figsize=(7,6))
plt.imshow(pmf_matrix, cmap="Blues", interpolation="nearest")

# Add values on each cell
for i in range(6):
    for j in range(6):
        plt.text(j, i, f"{pmf_matrix[i,j]:.3f}", ha='center', va='center', color='black')

plt.title("Joint Probability Mass Function (2 Dice)")
plt.xlabel("Y (Die 2)")
plt.ylabel("X (Die 1)")
plt.colorbar(label="Probability")
plt.xticks(np.arange(6), die_outcomes)
```

```
plt.yticks(np.arange(6), die_outcomes)
plt.show()
```



Compute sample means	$\bar{X}_1 = \frac{\sum X_1}{n_1}, \bar{X}_2 = \frac{\sum X_2}{n_2}$
Compute sample variances	$S_i^2 = \frac{\sum (x_i - \bar{x}_i)^2}{n_i - 1}$
Compute pooled variance	$S_p^2 = \frac{(n_1-1)S_1^2 + (n_2-1)S_2^2}{n_1+n_2-2}$
Compute t-statistic	$t = \frac{(\bar{X}_1 - \bar{X}_2) - (\mu_1 - \mu_2)}{S_p\sqrt{1/n_1 + 1/n_2}}$

Perform Large SampleTest for Single Mean(Z-test)

```
import numpy as np
from scipy.stats import norm

def z_test_single_mean(sample, mu0, sigma=None, alpha=0.05, alternative='two-sided'):

    sample = np.array(sample)
    n = len(sample)
    x_bar = np.mean(sample)

    if sigma is None:
        sigma = np.std(sample, ddof=1)

    # Z-statistic
    z_stat = (x_bar - mu0) / (sigma / np.sqrt(n))

    # p-value based on alternative hypothesis
    if alternative == 'two-sided':
        p_value = 2 * (1 - norm.cdf(abs(z_stat)))
    elif alternative == 'greater':
        p_value = 1 - norm.cdf(z_stat)
    elif alternative == 'less':
        p_value = norm.cdf(z_stat)
    else:
        raise ValueError("alternative must be 'two-sided', 'less', or 'greater'")

    # Decision
    reject = p_value <= alpha
```

```
    return {
        "Sample Mean": x_bar,
        "Hypothesized Mean": mu0,
        "Z-statistic": z_stat,
        "p-value": p_value,
        "Conclusion": "Reject H0" if reject else "Fail to Reject H0"
    }

# ----------------------------
# Example Usage
# ----------------------------
data = [52, 50, 53, 49, 48, 51, 54, 50, 52, 49]  # sample data
mu0 = 50   # hypothesized population mean
alpha = 0.05

result = z_test_single_mean(data, mu0, sigma=None, alpha=alpha, alternative='two-sided')
print(result)
```

```
#Small Sample t-test for Difference of Means
import math
from scipy.stats import t

# Data
sample1 = [12, 9, 13, 10, 8]
sample2 = [7, 10, 6, 9, 5]
alpha=0.05
n1, n2 = len(sample1), len(sample2)

# Means
mean1 = sum(sample1) / n1
mean2 = sum(sample2) / n2

# Variances
var1 = sum((x - mean1)**2 for x in sample1) / (n1 - 1)
var2 = sum((x - mean2)**2 for x in sample2) / (n2 - 1)

# Pooled standard deviation
sp = math.sqrt(((n1 - 1)*var1 + (n2 - 1)*var2) / (n1 + n2 - 2))

# t-statistic
t_value = (mean1 - mean2) / (sp * math.sqrt((1/n1) + (1/n2)))

# Degrees of freedom
df = n1 + n2 - 2

t_tab = t.ppf(1 - alpha/2, df)   # two-tailed test


print("Mean1 =", mean1)
print("Mean2 =", mean2)
print("Variance1 =", var1)
print("Variance2 =", var2)
print("Pooled SD =", sp)
print("t =", t_value)
print("Degrees of Freedom =", df)

if t_value > t_tab:
    print("\nReject Null Hypothesis (H0).")
    print("There is a significant difference between the two means.")
else:
    print("\nAccept Null Hypothesis (H0).")
    print("→ There is no significant difference between the two means.")
```

```
Mean1 = 10.4
Mean2 = 7.4
Variance1 = 4.3
Variance2 = 4.3
Pooled SD = 2.073644135332772
t = 2.28747855498907
Degrees of Freedom = 8

Accept Null Hypothesis (H0).
→ There is no significant difference between the two means.
```

$$z = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}}$$

```
#Perform Test for Single Proportion(LargeSample)
import math

# Step 1: Input data
n = int(input("Enter sample size (n): "))
x = int(input("Enter number of successes (x): "))
p0 = float(input("Enter assumed population proportion (p0): "))
alpha = float(input("Enter level of significance (e.g., 0.05): "))
tail = input("Enter test type (two/left/right): ").strip().lower()

# Step 2: Compute sample proportion
phat = x / n

# Step 3: Compute standard error
se = math.sqrt((p0 * (1 - p0)) / n)

# Step 4: Compute z statistic
z_cal = (phat - p0) / se

# Step 5: Determine critical (tabulated) z-value
if tail == "two":
    z_tab = 1.96 if alpha == 0.05 else 2.58 if alpha == 0.01 else 1.64
elif tail == "right":
    z_tab = 1.645 if alpha == 0.05 else 2.33
elif tail == "left":
    z_tab = -1.645 if alpha == 0.05 else -2.33
else:
    print("Invalid tail type! Please enter 'two', 'left', or 'right'.")
    exit()

# Step 6: Display results
print("\n--- RESULTS ---")
print(f"Sample proportion (p̂) = {phat:.3f}")
print(f"Population proportion (p0) = {p0}")
print(f"Calculated z = {z_cal:.3f}")
print(f"Tabulated z = {z_tab}")
print(f"Significance Level (α) = {alpha}")

# Step 7: Decision
if tail == "two":
    if abs(z_cal) > z_tab:
        print("\n Reject Null Hypothesis (H0).")
    else:
        print("\n Accept Null Hypothesis (H0).")
elif tail == "right":
    if z_cal > z_tab:
        print("\n Reject Null Hypothesis (H0).")
    else:
        print("\n Accept Null Hypothesis (H0).")
elif tail == "left":
    if z_cal < z_tab:
        print("\n Reject Null Hypothesis (H0).")
    else:
        print("\n Accept Null Hypothesis (H0).")
```

```
Enter sample size (n): 600
Enter number of successes (x): 157
Enter assumed population proportion (p0): 0.3
Enter level of significance (e.g., 0.05): 0.05
Enter test type (two/left/right): two

--- RESULTS ---
Sample proportion (p̂) = 0.262
Population proportion (p0) = 0.3
Calculated z = -2.049
Tabulated z = 1.96
Significance Level (α) = 0.05

 Reject Null Hypothesis (H0).
```

Double-click (or enter) to edit

```
#Perform F-test for Comparison of Variances (Small Samples)
import math
from scipy.stats import f

# Data
sample1 = [12, 9, 13, 10, 18]
sample2 = [7, 10, 6, 9, 15]
alpha=0.05
n1, n2 = len(sample1), len(sample2)

# Means
mean1 = sum(sample1) / n1
mean2 = sum(sample2) / n2

# Variances
var1 = sum((x - mean1)**2 for x in sample1) / (n1 - 1)
var2 = sum((x - mean2)**2 for x in sample2) / (n2 - 1)

print("Mean1 =", mean1)
print("Mean2 =", mean2)
print("Variance1 =", var1)
print("Variance2 =", var2)

if var1>var2:
  f_stat=var1/var2
  df1=len(sample1)-1
  df2=len(sample2)-1
else:
  f_stat=var2/var1
  df1=len(sample2)-1
  df2=len(sample1)-1
p_value=2*f.sf(f_stat, df1, df2)

if p_value<alpha:
  print("Reject Null Hypothesis (H0).")
  print("There is a significant difference between the two variances.")
else:
  print("Accept Null Hypothesis (H0).")
  print("There is no significant difference between the two variances.")
```

```
Mean1 = 12.4
Mean2 = 9.4
Variance1 = 12.3
Variance2 = 12.3
Accept Null Hypothesis (H0).
There is no significant difference between the two variances.
```

PerformFirst-OrderMarkovProcessSimulation

```
import numpy as np


states = [0, 1]   # 0 = State A, 1 = State B

# Transition Probability Matrix
P = [[0.1, 0.9],   # From A to [A, B]
     [0.8, 0.2]]   # From B to [A, B]


current_state = 0
n = 10  # number of steps

print("Markov Process Simulation:")
sequence = [current_state]

for i in range(n):
    current_state = np.random.choice(states, p=P[current_state])
    sequence.append(int(current_state))

print("Sequence of States:", sequence)
```

```
Markov Process Simulation:
Sequence of States: [0, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1]
```

Steady State Probability

```
import numpy as np

# Transition matrix
P = np.array([[0.7, 0.3],
              [0.4, 0.6]])

#vP = v , v(P - I) = 0
A = (P - np.eye(2)).T
b = np.zeros(2)

# Add the normalization condition: π1 + π2 = 1
A = np.vstack((A, np.ones(2)))
b = np.append(b, 1)
print(A)
print(b)
# Solve
steady_state = np.linalg.lstsq(A, b, rcond=None)[0]
print("Steady-state distribution:", steady_state)
```

```
[[-0.3  0.4]
 [ 0.3 -0.4]
 [ 1.   1. ]]
[0. 0. 1.]
Steady-state distribution: [0.57142857 0.42857143]
```

PerformSecond-OrderMarkovProcessSimulation

```
import numpy as np
import random

# Define states
states = [0, 1, 2]   # A, B, C

# Define transition probabilities for each pair (previous two states)
P = {
    (0, 0): [0.7, 0.2, 0.1],
    (0, 1): [0.3, 0.5, 0.2],
    (0, 2): [0.6, 0.2, 0.2],
    (1, 0): [0.6, 0.3, 0.1],
    (1, 1): [0.2, 0.7, 0.1],
    (1, 2): [0.3, 0.4, 0.3],
    (2, 0): [0.2, 0.5, 0.3],
    (2, 1): [0.3, 0.3, 0.4],
    (2, 2): [0.5, 0.2, 0.3]
}

# Initialize with two starting states
current_state = [random.choice(states), random.choice(states)]
sequence = current_state.copy()

n = 5  # number of steps

# Simulation loop
for i in range(n):
    prev_pair = (sequence[-2], sequence[-1])
    probs = P[prev_pair]
    next_state = np.random.choice(states, p=probs)
    sequence.append(next_state)

print("Second-Order Markov Process Simulation:")
print("Sequence of States:", sequence)
```

```
Second-Order Markov Process Simulation:
Sequence of States: [1, 0, np.int64(0), np.int64(0), np.int64(1), np.int64(2), np.int64(1)]
```

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

Karl Pearson's Coeffient of corelation

```
import numpy as np

# Sample data
x = np.array([5, 20, 30, 40, 50])
y = np.array([30, 50, 55, 60, 70])

# Calculate Pearson correlation coefficient
mean_x = np.mean(x)
mean_y = np.mean(y)

numerator = np.sum((x - mean_x) * (y - mean_y))
denominator = np.sqrt(np.sum((x - mean_x)**2) * np.sum((y - mean_y)**2))
r = numerator / denominator

print("Karl Pearson's Coefficient of Correlation (r):", round(r, 2))
```

```
Karl Pearson's Coefficient of Correlation (r): 0.98
```

$$\rho = 1 - \frac{6 \sum d_i^2}{n(n^2 - 1)}$$

Spearman's Rank Correlation

d_i = difference between ranks of each pair

n = number of observations

```
import numpy as np
from scipy.stats import rankdata

# Sample data
x = np.array([10, 20, 30, 40, 50])
y = np.array([10, 25, 30, 45, 55])

# Rank the data
rank_x = rankdata(x)
rank_y = rankdata(y)

# Calculate differences and Spearman's rho
d = rank_x - rank_y
n = len(x)
rho = 1 - (6 * np.sum(d**2)) / (n * (n**2 - 1))

print("Spearman's Rank Correlation (ρ):", round(rho, 4))
```

Regression Formula:

$Y = a + bX$

where slope of trend line is calculated as:

$$b_1 = \frac{\sum (x - \bar{x}) * (y - \bar{y})}{\sum (x - \bar{x})^2}$$

and the intercept is computed as:

$$b_0 = y - (b_1 * X)$$

b0 and b1 will determine the line of best fit

UseanydatasetandsolveSimpleLinearRegression

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Dataset
x = np.array([10, 20, 30, 40, 50])
y = np.array([25, 45, 65, 75, 95])

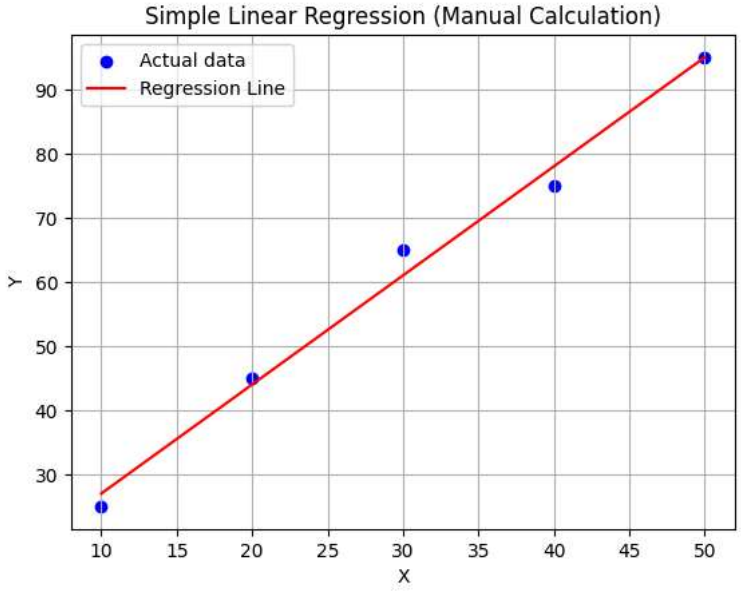
# Step 1: Calculate means
x_mean = np.mean(x)
y_mean = np.mean(y)

# Step 2: Calculate slope (b) and intercept (a)
numerator = np.sum((x - x_mean) * (y - y_mean))
denominator = np.sum((x - x_mean)**2)
b = numerator / denominator
a = y_mean - b * x_mean

print(f"Slope (b) = {b:.2f}")
print(f"Intercept (a) = {a:.2f}")
print(f"Equation of regression line: Y = {a:.2f} + {b:.2f}X")

# Step 3: Predicted values
y_pred = a + b * x
print(y_pred)
# Step 4: Plot
plt.scatter(x, y, color='blue', label='Actual data')
plt.plot(x, y_pred, color='red', label='Regression Line')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Simple Linear Regression (Manual Calculation)')
plt.legend()
plt.grid(True)
plt.show()
```

```
Slope (b) = 1.70
Intercept (a) = 10.00
Equation of regression line: Y = 10.00 + 1.70X
[27. 44. 61. 78. 95.]
```



UseanydatasetandsolveMultipleLinearRegression

$$Y = a + b_1X_1 + b_2X_2 + \cdots + b_nX_n$$

Y = dependent variable

$X_1, X_2, \ldots$ = independent variables

a = intercept

b_i = coefficients (slopes)

$$\boldsymbol{b} = (X^T X)^{-1} X^T Y$$

Here:

- X = matrix of inputs (with a column of 1s for the intercept)
- Y = output column vector
- $\boldsymbol{b}$ = coefficients vector $[a, b_1, b_2, \ldots]^T$

```
import numpy as np

# Dataset
X1 = np.array([230, 44, 17, 151, 180])
X2 = np.array([37, 39, 45, 41, 10])
Y  = np.array([22, 10, 9, 18, 13])

# Step 1: Form X matrix with a column of 1s for intercept
X = np.column_stack((np.ones(len(X1)), X1, X2))

# Step 2: Convert Y to column vector
Y = Y.reshape(-1, 1)

# Step 3: Apply Normal Equation: b = (X^T X)^(-1) X^T Y
b = np.linalg.inv(X.T @ X) @ X.T @ Y

# Step 4: Extract coefficients
a, b1, b2 = b.flatten()

print(f"Intercept (a): {a:.3f}")
print(f"Coefficient for (b1): {b1:.3f}")
print(f"Coefficient for  (b2): {b2:.3f}")
print(f"Regression Equation: Y = {a:.3f} + {b1:.3f}*TV + {b2:.3f}*Radio")

# Step 5: Predict for a new example
TV_new = 100
Radio_new = 30
Y_pred = a + b1 * TV_new + b2 * Radio_new
print(f"\nPredicted Sales for TV={TV_new}, Radio={Radio_new} → {Y_pred:.2f}")
```

```
Intercept (a): -1.528
Coefficient for TV (b1): 0.069
Coefficient for Radio (b2): 0.214
Regression Equation: Y = -1.528 + 0.069*TV + 0.214*Radio

Predicted Sales for TV=100, Radio=30 → 11.78
```

Lagrange Interpolation Formula

```
def lagrange_interpolation(x_values, y_values, x):
    n = len(x_values)
    result = 0.0

    for i in range(n):
        term = y_values[i]
        for j in range(n):
            if j != i:
                term *= (x - x_values[j]) / (x_values[i] - x_values[j])
        result += term
    return result

# Given data points
x_values = [5, 7, 11, 13]
y_values = [150, 392, 1452, 2366]

# Value to estimate
x_to_find = 10

# Apply Lagrange Interpolation
y_estimated = lagrange_interpolation(x_values, y_values, x_to_find)

print(f"Estimated value of f({x_to_find}) = {y_estimated:.2f}")
```

ImplementNewton’sInterpolationmethod

```
import numpy as np
import math

# Function to calculate divided differences
def forward_diff_table(y):
    n = len(y)
    diff_table = np.zeros((n, n))
    diff_table[:, 0] = y

    for j in range(1, n):
        for i in range(n - j):
            diff_table[i][j] = diff_table[i + 1][j - 1] - diff_table[i][j - 1]

    return diff_table

# Function to evaluate Newton polynomial at x_val
def newton_forward_interpolation(x, y, x_val):
    h = x[1] - x[0]  # uniform spacing
    diff_table = forward_diff_table(y)

    u = (x_val - x[0]) / h
    result = y[0]
    u_term = 1

    for i in range(1, len(y)):
        u_term *= (u - (i - 1))
        result += (u_term / math.factorial(i)) * diff_table[0][i]

    return result
# Given data
x = np.array([5, 7, 9, 11])
y = np.array([150, 392, 1452, 2366])

# Value to estimate
x_val = 10

# Apply Newton’s Interpolation
y_pred = newton_forward_interpolation(x, y, x_val)

print(f"Estimated value of f({x_val}) = {y_pred:.2f}")
```

```
Estimated value of f(10) = 1987.50
```

ApplyLUDecompositionforLinearSystems

```
import numpy as np

# Coefficient matrix A
A = np.array([[2, 3, 1],
              [4, 7, 5],
              [6, 18, 19]], dtype=float)

# Constant matrix B
B = np.array([1, 2, 3], dtype=float)

n = len(B)
L = np.zeros((n, n))
U = np.zeros((n, n))

# Step 1: Perform LU Decomposition (Doolittle's method)
for i in range(n):
    # Upper Triangular
    for k in range(i, n):
        sum_ = sum(L[i][j] * U[j][k] for j in range(i))
        U[i][k] = A[i][k] - sum_

    # Lower Triangular
    L[i][i] = 1
    for k in range(i + 1, n):
        sum_ = sum(L[k][j] * U[j][i] for j in range(i))
        L[k][i] = (A[k][i] - sum_) / U[i][i]
```

```
        L[k][i] = (A[k][i] - sum_) / U[i][i]

print("L (Lower Triangular Matrix):\n", L)
print("\nU (Upper Triangular Matrix):\n", U)

# Step 2: Solve L*y = B (Forward substitution)
```